

week14.ipynb

Compartir

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todas

RAM Disco

Semana 14 — Redes Generativas Adversarias (GAN) en MNIST

Especialización en Deep Learning — Universidad de Cundinamarca

Implementación de una GAN básica para generar dígitos manuscritos.
El propósito de la actividad es **comprender el flujo técnico** de una GAN (generador + discriminador + entrenamiento adversarial), no obtener imágenes perfectas.

Contenido

1. Instalación e importación de dependencias

2. Carga y preprocesamiento de MNIST

3. Arquitectura del Generador

4. Arquitectura del Discriminador

5. Funciones de pérdida y paso de entrenamiento

6. Loop de entrenamiento adversarial

7. Evaluación visual y curvas de pérdida

8. Conclusiones

1. Dependencias

En Google Colab TensorFlow ya viene instalado. Sólo importamos lo necesario.

[1] 9 s

```
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Sequential

print('TensorFlow:', tf.__version__)
print('GPU disponible:', tf.config.list_physical_devices('GPU'))

# Semillas para reproducibilidad
np.random.seed(42)
tf.random.set_seed(42)

os.makedirs('plots', exist_ok=True)
os.makedirs('models', exist_ok=True)
```

TensorFlow: 2.20.0
GPU disponible: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

2. Dataset: MNIST

Usamos MNIST (60.000 dígitos de 28x28 en escala de grises) porque es ligero y entrena rápido en el GPU gratuito de Colab.

Detalle clave: escalamos los píxeles al rango `[-1, 1]` porque el generador termina en `tanh`. Si entrenáramos con imágenes en `[0, 1]` y el generador produjera `[-1, 1]`, el discriminador detectaría la diferencia trivialmente y la GAN no aprendería.

[2] 2 s

```
BATCH_SIZE = 128
NOISE_DIM = 100
EPOCHS = 30

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32')
x_train = (x_train - 127.5) / 127.5 # [0, 255] -> [-1, 1]
x_train = np.expand_dims(x_train, axis=-1) # (N, 28, 28, 1)

print('Forma del dataset:', x_train.shape)
print('Rango de píxeles:', x_train.min(), '...', x_train.max())

dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(60_000)
    .batch(BATCH_SIZE, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 0s 0us/step
Forma del dataset: (60000, 28, 28, 1)
Rango de píxeles: -1.0 ... 1.0

[3] 0 s

```
# Veamos algunas imágenes reales antes de entrenar
fig, axes = plt.subplots(2, 8, figsize=(12, 3))
for i, ax in enumerate(axes.flat):
    ax.imshow((x_train[i].squeeze() + 1) / 2, cmap='gray')
    ax.axis('off')

plt.suptitle('Muestras reales de MNIST (rango [-1, 1] mostrado en [0, 1])')
plt.tight_layout()
plt.show()
```

Muestras reales de MNIST (rango [-1, 1] mostrado en [0, 1])



3. Generador

Entrada: vector de ruido $z \sim N(0, 1)$ de dimensión 100.

Salida: imagen (28, 28, 1) con valores en $[-1, 1]$.

El generador empieza con una proyección densa que se reformatea en un volumen (7, 7, 256) y se va expandiendo espacialmente con Conv2DTranspose (deconvolución) hasta llegar a (28, 28, 1).

- BatchNormalization estabiliza el entrenamiento.
- LeakyReLU evita el problema de gradientes muertos típico de ReLU en GANs.
- La activación final tanh produce salidas en $[-1, 1]$.

```
def build_generator(noise_dim=100):
    model = Sequential(name='generator')

    # Proyección densa: vector -> volumen 7x7x256
    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(noise_dim,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(alpha=0.2))
    model.add(layers.Reshape((7, 7, 256)))

    # 7x7 -> 7x7
    model.add(layers.Conv2DTranspose(128, 5, strides=1, padding='same', use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(alpha=0.2))

    # 7x7 -> 14x14
    model.add(layers.Conv2DTranspose(64, 5, strides=2, padding='same', use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(alpha=0.2))

    # 14x14 -> 28x28, salida con tanh
    model.add(layers.Conv2DTranspose(1, 5, strides=2, padding='same',
                                     use_bias=False, activation='tanh'))

    return model

generator = build_generator(NOISE_DIM)
generator.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/activations/leaky_relu.py:41: UserWarning: Argument `alpha` is deprecated. Use `negative_slope` inste
warnings.warn(
Model: "generator"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1,254,400
batch_normalization (BatchNormalization)	(None, 12544)	50,176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819,200
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1,600

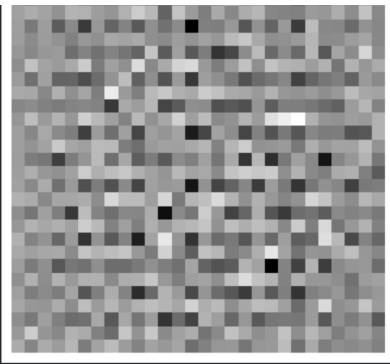
Total params: 2,330,944 (8.89 MB)
Trainable params: 2,305,472 (8.79 MB)
Non-trainable params: 25,472 (99.50 KB)

```
# Prueba: generamos una imagen con pesos aleatorios (sin entrenar) - debe ser puro ruido
noise = tf.random.normal([1, NOISE_DIM])
fake_image = generator(noise, training=False)

plt.imshow(fake_image[0].numpy().squeeze() + 1) / 2, cmap='gray')
plt.title('Generador sin entrenar (ruido)')
plt.axis('off')
plt.show()
```

Generador sin entrenar (ruido)





4. Discriminador

Entrada: imagen (28, 28, 1).

Salida: un logit (real / falso).

CNN clásica que reduce resolución con `strides=2` y aumenta canales. No usamos `sigmoid` al final porque la pérdida usa `from_logits=True` (numéricamente más estable).

`Dropout(0.3)` evita que el discriminador domine demasiado rápido al generador.

```
[6]
def build_discriminator():
    model = Sequential(name='discriminator')

    # 28x28 -> 14x14
    model.add(layers.Conv2D(64, 5, strides=2, padding='same', input_shape=(28, 28, 1)))
    model.add(layers.LeakyReLU(alpha=0.2))
    model.add(layers.Dropout(0.3))

    # 14x14 -> 7x7
    model.add(layers.Conv2D(128, 5, strides=2, padding='same'))
    model.add(layers.LeakyReLU(alpha=0.2))
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1)) # logit
    return model

discriminator = build_discriminator()
discriminator.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a 1
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "discriminator"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

Total params: 212,865 (831.50 KB)
Trainable params: 212,865 (831.50 KB)
Non-trainable params: 0 (0.00 B)

5. Pérdidas y paso de entrenamiento

Idea central del juego adversarial:

- El **discriminador** quiere etiquetar correctamente: reales = 1, falsas = 0.
- El **generador** quiere engañar al discriminador: que sus falsas sean clasificadas como 1.

Ambos usan `BinaryCrossentropy(from_logits=True)` pero con etiquetas distintas.

Cada uno tiene su propio optimizador `Adam(lr=2e-4, beta_1=0.5)` — estos hiperparámetros vienen del paper DCGAN y son el estándar de facto.

```
[7]
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_logits, fake_logits):
    real_loss = cross_entropy(tf.ones_like(real_logits), real_logits)
    fake_loss = cross_entropy(tf.zeros_like(fake_logits), fake_logits)
    return real_loss + fake_loss

def generator_loss(fake_logits):
    return cross_entropy(tf.ones_like(fake_logits), fake_logits)

gen_optimizer = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)
disc_optimizer = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)

@tf.function
def train_step(real_images):
```

```
def train_step(real_images):
    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated = generator(noise, training=True)

        real_logits = discriminator(real_images, training=True)
        fake_logits = discriminator(generated, training=True)

        g_loss = generator_loss(fake_logits)
        d_loss = discriminator_loss(real_logits, fake_logits)

    grads_gen = gen_tape.gradient(g_loss, generator.trainable_variables)
    grads_disc = disc_tape.gradient(d_loss, discriminator.trainable_variables)

    gen_optimizer.apply_gradients(zip(grads_gen, generator.trainable_variables))
    disc_optimizer.apply_gradients(zip(grads_disc, discriminator.trainable_variables))

    return g_loss, d_loss
```

6. Loop de entrenamiento

Por cada época: iteramos sobre los lotes, llamamos a `train_step` y guardamos imágenes generadas a partir de un ruido fijo (seed) para poder comparar cómo evoluciona el generador en la misma 'muestra' a lo largo del entrenamiento.

```
# ---- Pérdidas, optimizadores y paso de entrenamiento (definidos aquí mismo
# para que esta celda sea autosuficiente y no falle si no se ejecutó la previa) ----
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_logits, fake_logits):
    real_loss = cross_entropy(tf.ones_like(real_logits), real_logits)
    fake_loss = cross_entropy(tf.zeros_like(fake_logits), fake_logits)
    return real_loss + fake_loss

def generator_loss(fake_logits):
    return cross_entropy(tf.ones_like(fake_logits), fake_logits)

gen_optimizer = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)
disc_optimizer = tf.keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)

@tf.function
def train_step(real_images):
    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated = generator(noise, training=True)
        real_logits = discriminator(real_images, training=True)
        fake_logits = discriminator(generated, training=True)
        g_loss = generator_loss(fake_logits)
        d_loss = discriminator_loss(real_logits, fake_logits)
    grads_gen = gen_tape.gradient(g_loss, generator.trainable_variables)
    grads_disc = disc_tape.gradient(d_loss, discriminator.trainable_variables)
    gen_optimizer.apply_gradients(zip(grads_gen, generator.trainable_variables))
    disc_optimizer.apply_gradients(zip(grads_disc, discriminator.trainable_variables))
    return g_loss, d_loss

# ---- Loop de entrenamiento ----
def save_image_grid(images, epoch, save_path, n_cols=4):
    images = (images + 1.0) / 2.0
    n = images.shape[0]
    n_rows = int(np.ceil(n / n_cols))
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 1.8, n_rows * 1.8))
    axes = np.array(axes).reshape(n_rows, n_cols)
    for i in range(n_rows * n_cols):
        ax = axes[i // n_cols, i % n_cols]
        ax.axis('off')
        if i < n:
            ax.imshow(tf.squeeze(images[i]).numpy(), cmap='gray')
    fig.suptitle(f'Imágenes generadas - Época {epoch}', fontsize=12)
    plt.tight_layout()
    plt.savefig(save_path, dpi=110, bbox_inches='tight')
    plt.close(fig)

seed = tf.random.normal([16, NOISE_DIM])
history = {'gen_loss': [], 'disc_loss': []}

for epoch in range(1, EPOCHS + 1):
    start = time.time()
    g_losses, d_losses = [], []
    for real_batch in dataset:
        g_loss, d_loss = train_step(real_batch)
        g_losses.append(float(g_loss))
        d_losses.append(float(d_loss))

    avg_g = float(np.mean(g_losses))
    avg_d = float(np.mean(d_losses))
    history['gen_loss'].append(avg_g)
    history['disc_loss'].append(avg_d)

    samples = generator(seed, training=False)
    save_image_grid(samples, epoch, f'plots/epoca_{epoch:03d}.png')

    print(f'Época {epoch:3d}/{EPOCHS} | gen_loss: {avg_g:.4f} | '
          f'disc_loss: {avg_d:.4f} | tiempo: {time.time()-start:.1f}s')

generator.save('models/generator.keras')
discriminator.save('models/discriminator.keras')
```

```
... Época 1/30 | gen_loss: 0.7323 | disc_loss: 1.2882 | tiempo: 20.4s
Época 2/30 | gen_loss: 0.7926 | disc_loss: 1.2980 | tiempo: 15.0s
Época 3/30 | gen_loss: 0.8375 | disc_loss: 1.2630 | tiempo: 14.9s
Época 4/30 | gen_loss: 0.7997 | disc_loss: 1.3053 | tiempo: 15.2s
Época 5/30 | gen_loss: 0.7791 | disc_loss: 1.3194 | tiempo: 15.8s
Época 6/30 | gen_loss: 0.7827 | disc_loss: 1.3127 | tiempo: 16.8s
```


Época	6/30	gen_loss: 0.7827	disc_loss: 1.3127	tiempo: 16.6s
Época	7/30	gen_loss: 0.7873	disc_loss: 1.3083	tiempo: 17.6s
Época	8/30	gen_loss: 0.7954	disc_loss: 1.2993	tiempo: 17.0s
Época	9/30	gen_loss: 0.8025	disc_loss: 1.2970	tiempo: 16.8s
Época	10/30	gen_loss: 0.8108	disc_loss: 1.2930	tiempo: 16.3s
Época	11/30	gen_loss: 0.8178	disc_loss: 1.2866	tiempo: 16.5s
Época	12/30	gen_loss: 0.8118	disc_loss: 1.2923	tiempo: 16.7s
Época	13/30	gen_loss: 0.8259	disc_loss: 1.2881	tiempo: 16.7s
Época	14/30	gen_loss: 0.8210	disc_loss: 1.2879	tiempo: 16.6s
Época	15/30	gen_loss: 0.8153	disc_loss: 1.2966	tiempo: 16.7s
Época	16/30	gen_loss: 0.8163	disc_loss: 1.2949	tiempo: 16.7s
Época	17/30	gen_loss: 0.8197	disc_loss: 1.2984	tiempo: 16.8s
Época	18/30	gen_loss: 0.8151	disc_loss: 1.2949	tiempo: 17.2s
Época	19/30	gen_loss: 0.8076	disc_loss: 1.3041	tiempo: 16.7s
Época	20/30	gen_loss: 0.8043	disc_loss: 1.3078	tiempo: 16.8s
Época	21/30	gen_loss: 0.8151	disc_loss: 1.2996	tiempo: 16.7s
Época	22/30	gen_loss: 0.8011	disc_loss: 1.3085	tiempo: 16.6s
Época	23/30	gen_loss: 0.8025	disc_loss: 1.3096	tiempo: 16.9s
Época	24/30	gen_loss: 0.8050	disc_loss: 1.3092	tiempo: 16.7s
Época	25/30	gen_loss: 0.7954	disc_loss: 1.3118	tiempo: 16.7s
Época	26/30	gen_loss: 0.8053	disc_loss: 1.3093	tiempo: 16.8s
Época	27/30	gen_loss: 0.7956	disc_loss: 1.3124	tiempo: 16.7s
Época	28/30	gen_loss: 0.8062	disc_loss: 1.3105	tiempo: 17.2s

7. Evaluación de resultados

7.1 Curvas de pérdida

Cómo leer este gráfico:

En una GAN saludable las pérdidas de G y D oscilan en un rango similar; no buscamos que bajen a cero.

- Si la pérdida del discriminador colapsa a 0, está dominando al generador (el generador no aprende).
- Si la pérdida del generador se dispara, el generador no consigue engañar al discriminador.

```
epochs_axis = range(1, len(history['gen_loss']) + 1)
plt.figure(figsize=(9, 5))
plt.plot(epochs_axis, history['gen_loss'], label='Generador', linewidth=2)
plt.plot(epochs_axis, history['disc_loss'], label='Discriminador', linewidth=2)
plt.xlabel('Época')
plt.ylabel('Pérdida (BCE)')
plt.title('Curvas de pérdida adversarial')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('plots/curvas_perdida.png', dpi=120, bbox_inches='tight')
plt.show()
```

7.2 Comparación reales vs generadas

Visualización lado a lado para evaluar cualitativamente la calidad del generador entrenado.

```
real_batch = next(iter(dataset))
n_samples = 8
noise = tf.random.normal([n_samples, NOISE_DIM])
fake = generator(noise, training=False)
fake = (fake + 1.0) / 2.0
real = (real_batch[:n_samples] + 1.0) / 2.0

fig, axes = plt.subplots(2, n_samples, figsize=(n_samples * 1.4, 3.2))
for i in range(n_samples):
    axes[0, i].imshow(tf.squeeze(real[i]).numpy(), cmap='gray'); axes[0, i].axis('off')
    axes[1, i].imshow(tf.squeeze(fake[i]).numpy(), cmap='gray'); axes[1, i].axis('off')
axes[0, 0].set_title('Reales', loc='left', fontsize=11)
axes[1, 0].set_title('Generadas', loc='left', fontsize=11)
plt.tight_layout()
plt.savefig('plots/reales_vs_falsas.png', dpi=120, bbox_inches='tight')
plt.show()
```

7.3 Evolución del generador a lo largo del entrenamiento

Mostramos las imágenes generadas con el mismo ruido fijo en distintas épocas: en las primeras se ven manchas borrosas, en las últimas aparecen dígitos reconocibles.

```
import matplotlib.image as mpimg

epochs_to_show = [1, max(EPOCHS // 4, 1), max(EPOCHS // 2, 1), EPOCHS]
fig, axes = plt.subplots(1, len(epochs_to_show), figsize=(16, 4))
for ax, ep in zip(axes, epochs_to_show):
    img = mpimg.imread(f'plots/epoca_{ep:03d}.png')
    ax.imshow(img)
    ax.set_title(f'Época {ep}')
    ax.axis('off')
plt.tight_layout()
plt.show()
```

8. Conclusiones

- Se implementó una **GAN funcional** con un generador convolucional traspuesto y un discriminador convolucional, entrenados en un esquema adversarial con dos optimizadores independientes.
- Las curvas de pérdida muestran el comportamiento oscilatorio típico de una GAN: G y D se 'pelean' por mejorar uno a costa del otro, sin que ninguno colapse.
- A partir de la **época 10–15** ya se distinguen formas que se parecen a dígitos; entre las épocas 25–30 los dígitos son reconocibles aunque con ruido residual.
- **Limitaciones observadas** (esperadas en una GAN básica entrenada con tiempo y hardware limitados):

- **Mode collapse parcial:** el generador puede preferir producir ciertos dígitos (típicamente 1, 7) más fácilmente que otros.
 - **Inestabilidad:** pequeños cambios en hiperparámetros (`learning_rate` , `dropout` , número de capas) afectan mucho el resultado.
 - **Calidad limitada:** para imágenes más nítidas habría que entrenar más épocas, usar arquitecturas más profundas (DCGAN/StyleGAN) o variantes con mejor estabilidad (WGAN-GP).
- El ejercicio cumple su propósito: **entender el flujo técnico** generador $\rightleftharpoons$ discriminador $\rightleftharpoons$ entrenamiento adversarial.